Documentation for Simplex Codes

By: Immanuel Manohar
(idmanohar@crimson.ua.edu)

For: MA520 – Linear Optimization Final Project

Implemented Algorithms:

1. Two Phase Simplex
2. Revised Simplex
3. Dual Simplex
4. Transportation Problem
5. Unbalanced Transportation Problem

The codes are commented when necessary and displays all the steps in the iteration procedure.

General Information:

The following functions are created to address all the problems:

Function Name	Role
Simplex	Primary function that helps obtain and store variables as .mat files to be used later. This accepts either standard simplex, transportation or transshipment problems. (This is the starting function to run) Eg: simplex(1) to solve a standard simplex., simplex(a,'fname.mat') to use stored values in fname.mat to run simplex.
Simplex1	Function used to separate uses of Two Phase / Dual / Revised / Primal Dual Simplex algorithms
Two_phase_simplex	Function that solves two phase simplex, returns optimal primal variables X and cost Z.
Simplex_iteration	Function block that helps perform the iterations in two phase simplex.
Revised_simplex	Function that solves revised simplex, returns optimal primal variables X and cost Z.
Revised_simplex_iteration	Function block that helps perform the iterations in two revised simplex.
Dual_simplex	Function that helps solve Dual simplex problems.
Simplex_transportation	Function block that helps solve both balanced and Unbalanced transportation problems.
Simplex_balancedtransportation	Function that helps solve balanced transportation problems (called from simplex_transportation)
Cycle	A function block that assists in finding the next basic feasible solution with the new variable added. In solving balanced or unbalanced transportation problem.
Example_1.mat, Example_2.mat, Example_3.mat, Example_4.mat, Example_5.mat	.mat files with preentered values for illustrating the various simplex algorithms. The examples are detailed further in this report. The examples are based off the book Linear and Non-linear programming by Luenberger.

Examples Used:

Example 1: (Taken from example 2 pg 51 of Linear and non-linear programming by Luenberger) Used to illustrate both Two phase and Revised Simplex.

$$\begin{aligned}&\min 4x_1 + x_2 + x_3\\ &st, 2x_1 + x_2 + 2x_3 = 4\\ &3x_1 + 3x_2 + x_3 = 3\\ &x_1, x_2, x_3 \geq 0\end{aligned}$$

Implement by calling : **simplex(1)** (or) **simplex(1,Example_1.mat**) in MATLAB

The same is used as example 2 for revised simplex.

Implement by calling : **simplex(1)** (or) **simplex(1,Example_2.mat)** in MATLAB

Example 3: Dual simplex illutstration: (Taken from Pg 101 of Book)

$$\begin{aligned}&\min 3x_1 + 4x_2 + 5x_3\\ &ST: x_1 + 2x_2 + 3x_3 \geq 5\\ &2x_1 + 2x_2 + x_3 \geq 6\\ &x_1, x_2, x_3 \geq 0\end{aligned}$$

Implement by calling : **simplex(1)** (or) **simplex(1,Example_3.mat)** in MATLAB

Example 4: Balanced Transportation Problem (Book: Pg 57)

$$a = [30, 80, 10, 60]$$
$$b = [10, 50, 20, 80, 20]$$
$$C = \begin{bmatrix}3 & 4 & 6 & 8 & 9\\ 2 & 2 & 4 & 5 & 5\\ 2 & 2 & 2 & 3 & 2\\ 3 & 3 & 2 & 4 & 2\end{bmatrix}$$

Implement by calling : **simplex(2)** (or) **simplex(1,Example_4.mat)** in MATLAB

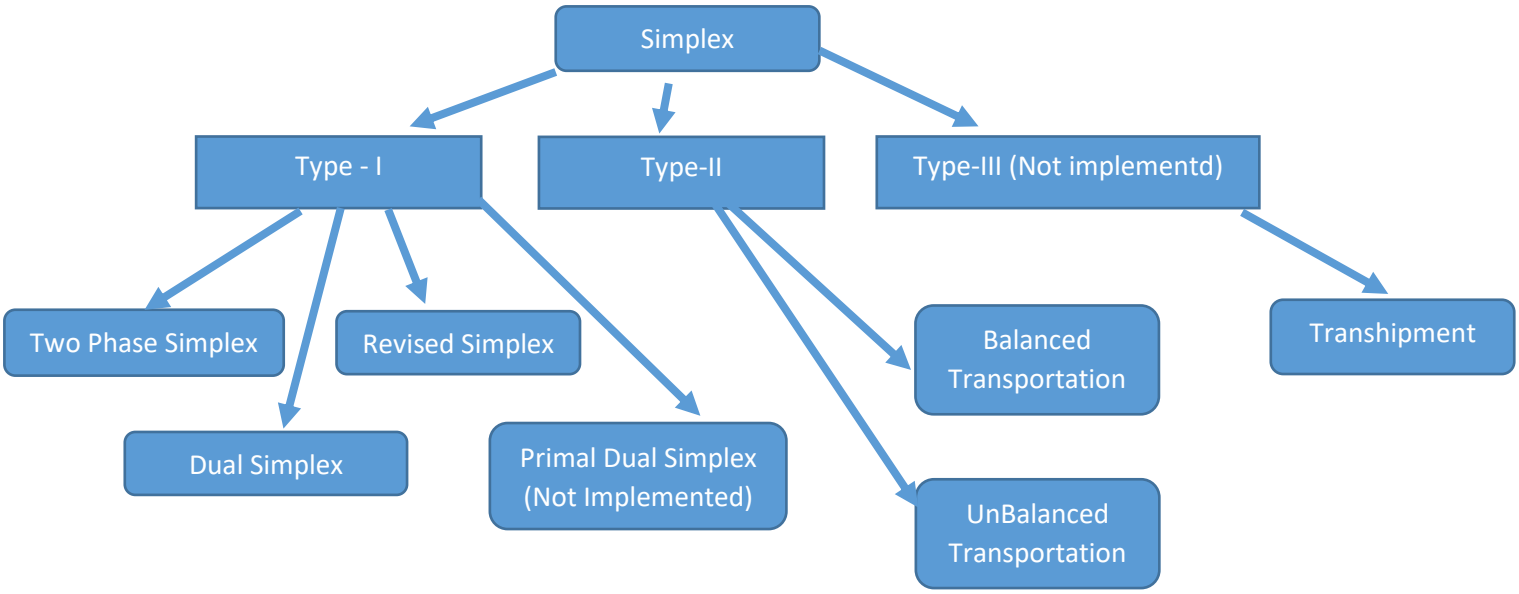
Example 5: UnBalanced Transportation Problem

$$a = [30, 80, 10, 60]$$
$$b = [10, 50, 20, 80]$$
$$C = \begin{bmatrix}4 & 6 & 8 & 9\\ 2 & 4 & 5 & 5\\ 2 & 2 & 3 & 2\\ 3 & 2 & 4 & 2\end{bmatrix}$$

Implement by calling : **simplex(2)** (or) **simplex(1,Example_5.mat)** in MATLAB

The MATLAB Codes:

The functions are organized as below:



The following function is used to start solving any simplex problem belonging to the 5 examples illustrated above:

Universal Functions:

```
%*************************************************************************
%Function coded by Immanuel Manohar (idmanohar@crimson.ua.edu)
%*************************************************************************
%function designed for solving standard linear programming problems:
%solves the three types of problems  (1. Standard Linear programming,
%2. Transportaion and 3. Transshipment) using 7 differnt algorithms. Details are given
below:
%
%The funciton could be called simply as:
%simplex(Type) where Type is 1 / 2 / 3 based on above.
%
%or if you want to use a .mat file with preentered values,
%
%simplex(Type,'file_name') In this the Type does not matter and will be
%overwritten from the file if present.
%
% This function returns a saved matfile with the olutions.
% The solutions are in variables X ->  The optimal solution if
% exists, else returns all possible Basic feasible solutions or returns NA
% if  infeasible.
% Z -> the optimal cost if exists or gives NA.
%
```

```
%
%Type 1:
%standard linear program:
%min c'x
%st Ax = b
% x >=0.
%
%solvers:
%  'TPS' : two phase simplex
%  'Revised' : Revised simplex
%  'DS': Dual simplex algorithm
%  'PD': Primal Dual Algorithm
%
%Type 2:
%network flow problem using graphs.
% C = cost of flow between nodes.
% bi = outflow - inflow at each node i.
% bi >0 for source node,
% bi < 0  for sink and
% bi = 0 for relay nodes.
%
%Type 3:
% Transportation problem
% C = cost of flow between nodes.
% a: quantity at source
% b: demand
% S: storage costs if any.

function  simplex(Type,varargin)
clc;
if isempty(varargin)== 1
switch Type
    case 1
        reenter =1;
        while reenter == 1
        disp('You are trying to solve a standard Linear Program')
        disp('Enter the input parametrs')
        disp('-------------------------')
        c = input('Enter the cost values,c: ');
        c = c(:)';
        A = input('Enter the constraint matrix A: ');
        b = input('Enter the right side of the constraint equation, b: ');
        b= b(:);
        solver = input('Enter solver to be used:\\n (TP- Two phase simplex), (Revised
- Revised Simplex), (DS - Dual Simplex),\\n (PD - Primal Dual Algorithm), Your Choice:
','s');
        disp('Input parameters are entered')
        disp('---------------------------')
        disp(sprintf('solution to be found using %s:',solver));
        disp('The cost c is given by: ');
        c
        disp('The constraint matrix A:');
        A
        disp('The right side of the constraint equation: b');
        b
        store = input('confirm above and store values? (1 = yes, 0 = no)');
        if store ==1
            name1 = input('enter file name to store as:' ,'s');
            name = sprintf('%s.mat',name1);
            save(name,'c','A','b','Type','solver','name');
            reenter = 0;
            [X,Z,D] = simplex1(name);
            disp('-------------------------')
```

```
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            disp('The dual is: ')
            D
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
            save(name,'c','A','b','Type','solver','X','Z','name');
        else
            reenter =1;
        end
        end
    case 2
        reenter = 1;
        while reenter == 1
            disp('You are trying to solve a transportation problem')
        disp('Enter the input parametrs')
        disp('-------------------------')
        C = input('Enter the cost matrix,c: ');
        a = input('Enter the source quantities: ');
        b = input('Enter the destination quantities: ');
        S = input('Enter the storage cost (if any): ');

        disp('Input parameters are entered')
        disp('----------------------------')
        disp('solution to be found using Array format of simplex based on northwest
corner rule');
        disp('The cost c is given by: ');
        C
        disp('The source quantities is given by:');
        a
        disp('The Destination quantities is given by:');
        b
        disp('The Storage cost is given by:')
        S
        store = input('confirm above and store values? (1 = yes, 0 = no)');
        if store ==1
            name1 = input('enter file name to store as:','s');
            name = sprintf('%s.mat',name1);
            save(name,'C','a','b','S','Type','name');
            reenter = 0;
            [X,Z] = simplex_transportaion(name);
            disp('-------------------------')
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
            save(name,'C','a','b','S','Type','X','Z','name');
        else
            reenter = 1;
        end
        end
    case 3
        reenter = 1;
        while reenter == 1
        disp('You are trying to solve a network flow problem')
        disp('Enter the input parametrs')
        disp('-------------------------')
```

```
        C = input('Enter the cost matrix,c: ');
        b = input('Enter b, the node demand: ');


        disp('Input parameters are entered')
        disp('----------------------------')
        disp('solution to be found using Network format for Simplex');
        disp('The cost c is given by:');
        C
        disp('Demands of each node is given by:');
        b

        store = input('confirm above and store values?');
        if store ==1
            name1 = input('enter file name to store as:','s');
            name = sprintf('%s.mat',name1);
            save(name,'C','b','Type','name');
            reenter = 0;
            [X,Z] = simplex_network(name);
            disp('-------------------------')
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
            save(name,'C','b','Type','X','Z','name');
        end
        end
    otherwise
        error('Enter a valid Type (1/2/3)')

end

else
    fname = varargin{1}
    load(fname);

    switch Type
        case 1
             [X,Z,D] = simplex1(name);
            disp('-------------------------')
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            disp('The dual is: ')
            D
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
            save(name,'c','A','b','Type','solver','X','Z','name');
        case 2
            [X,Z] = simplex_transportaion(name);
            disp('-------------------------')
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
```

```
            save(name,'C','a','b','Type','X','Z','name');
        case 3
            [X,Z] = simplex_network(name);
            disp('-------------------------')
            disp ('The optimal basic feasible solution(s) is: ')
            X
            disp('-------------------------')
            disp (sprintf('The optimal cost is Z : %d',Z));
            Z
            fprintf('---------(Results are also stored in the file name: %s)----------
------',name)
            save(name,'c','b','Type','X','Z','name');
        otherwise
            error('The type in file does not exist');
    end

end
```

The function simplex1 is used to select which simplex algorithm to use to solve the standard simplex model.:

```
%Code by Immanuel Manohar (idmanohar@crimson.ua.edu)
%selects the algorithm to solve the Stabndard Linear Programming
function [X,Z,D] = simplex1(fname)

load(fname)

switch solver
    case 'TP'
        [X,Z] = Two_phase_simplex(fname);
        D = 'NA';

    case 'Revised'
        [X,Z,D] = Revised_simplex(fname);

    case 'DS'
%         error('Will be implemented later');
        [X,Z] = Dual_simplex(fname);
        D = 'NA';
    case 'PD'
        error('Will be implemented later');
        %[X,Z,D] = Primal_Dual(fname);
end
```

Two Phase simplex:

The two phase simplex algorithm has two function blocks.

Part – 1

This following function block determines the parameters for phase I and phase II and calls the simplex_iterations.m function to solve the LP in both Phase I and Phase =II.

```
function [X,Z] = Two_phase_simplex(fname)

load(fname)
```

```
n = size(A,2);
fprintf('Number of optimization variables: %d \n', n )
m = size(A,1);
fprintf('Number of basic variables: %d \n', m )

%% Phase - I finding the initial basic feasible solution

% creating new A with dummy variables
b= b(:);
c = c(:)';
A_new = [A eye(size(A,1))];

c_new = [zeros(1,size(A,2)) ones(1,size(A,1))];

% b remains unaltered.

% simplex_iteration with new A and cost to obtain initial BFS.
basic_variables = zeros(1,size(A_new,2));
basic_variables(size(A,2)+1:end)= ones(1,size(A,1)); %the dummy variables form the
initial basic feasible solution in Phase I.
disp('Phase I to determine initial basic feasible solution ')
[A_return, b_return,basic_variables,~,~,flag] =
simplex_iteration(A_new,b,c_new,basic_variables);
if flag == 0
    X = 'NA';
    Z = 'NA';
else
    disp('Phase II to determine optimal basic feasible solution ')
    A_refined = A_return(1:size(A,1), 1: size(A,2));
    b_refined = b_return;
    c_refined = c;
    [A, b,basic_variables,Z,~,flag] =
simplex_iteration(A_refined,b_refined,c_refined,basic_variables);
    if flag == 0
        X = 'NA';
        Z = 'NA';

    else
        % Find columns of A that are basic.
        X = zeros(1,size(A,2));
        for i = 1 : size(A,2)
            if basic_variables(i) == 1
                X(i) = b(A(:,i) == 1);
            end
        end
        disp('******************************************************* ')
        display(sprintf(' The optimal solution is given by: X : %s \n',num2str(X)))
        display(sprintf(' The optimal objective value is given by: Z : %d \n',Z))
        disp('*******************************************************')

    end
end
```

Part – 2

The simplex iterations are solved using the function below:

```
%Code by Immanuel Manohar (idmanohar@crimson.ua.edu)
```

```
function [A_return, b_return, basic_variables,  z, r, flag] =
simplex_iteration(A,b,c,basic_variables)
flag = 1;
z = 0;
r = c;
%forming initial tableau:

Tableau = [A b;r 0];

disp('The initial simplex tableau is given by: ')
Tableau

%Find the columns corresponding to the initial basic feasible solution and
%setting the corresponding cost to zero.
bs_no = 0;
for i = 1: size(A,2)
  if (norm(A(:,i),1) == norm(A(:,i),Inf)) && (norm(A(:,i),1) == 1)
      %i is a column corresponding to basic feasible solution.
      bs_no = bs_no+1;
      pivot_column = i;
      pivot_row = A(:,i)== 1;

      if r(i) ~= 0
          Tableau(end,:) = Tableau(end,:)-Tableau(pivot_row,:).*r(i);
      end
  end
end

if bs_no ~= size(A,1)
    error('Need initial basic feasible solution to start simplex iteration ');
end

disp(sprintf('The 1st tableau after transforming the last row is: '))
Tableau

Iteration = 1;
while (Iteration < 20)
    r = Tableau(end,1:end-1);
    A = Tableau(1:end-1,1:end-1);
    b = Tableau(1:end-1,end);
    z = -Tableau(end,end);
    if sum(r < 0) == 0
        display('All relative cost coeffitients > 0, optimal solution reached')
        A_return = A;
        b_return = b;
        return
    else
        Iteration = Iteration +1;
        [~,Id] = min(r);
        pivot_column = Id(1); %incase there are multiple r(i) with same minimum value.
        basic_variables(pivot_column) = 1;
        ratio = b./A(:,pivot_column);
        if sum((ratio >= 0)) ==0
            display('The problem is unbounded')
            A_return = A;
            b_return = b;
            flag = 0;
            return
        end

        [~,Id] = min((ratio > 0).*ratio + (ratio <= 0).*Inf);
        pivot_row = Id(1); %incase there are multiple ratios with the same minimum
value >0;
```

```
        basic_variables(A(pivot_row,:)== 1) = 0;
        fprintf('Pivot Row: %d; Pivot Column: %d \n',pivot_row,pivot_column)
        %pivoting with the found pivot element:
        Tableau(pivot_row,:) = Tableau(pivot_row,:)./Tableau(pivot_row,pivot_column);
        temp = Tableau(pivot_row,:);
        Tableau = Tableau -
diag(Tableau(:,pivot_column))*ones(size(Tableau))*diag(temp);
        Tableau(pivot_row,:) = temp;
        disp(sprintf('The %d tableau is: \n', Iteration))
        Tableau
    end
end
```

Revised Simplex:

This also has two functions similar to the previous one. The first block below initiates the different Phases, the Second block solves the simplex algorithm.

Part –1

```
%Code by Immanuel Manohar (idmanohar@crimson.ua.edu)

function [X,Z,D] = Revised_simplex(fname)
load(fname)

n = size(A,2);
fprintf('Number of optimization variables: %d \n', n )
m = size(A,1);
fprintf('Number of basic variables: %d \n', m )

%% Phase - I finding the initial basic feasible solution

% creating new A with dummy variables
b= b(:);
c = c(:)';
A_new = [A eye(size(A,1))];

c_new = [zeros(1,size(A,2)) ones(1,size(A,1))];

% b remains unaltered.
% simplex_iteration with new A and cost to obtain initial BFS.
basic_variables = zeros(1,size(A_new,2));
basic_variables(size(A,2)+1:end)= ones(1,size(A,1)); %the dummy variables form the
initial basic feasible solution in Phase I.
disp('Phase I to determine initial basic feasible solution ')
[A_return, b_return,basic_variables,~,flag_new] =
revised_simplex_iteration(A_new,b,c_new,basic_variables);
if flag_new == 0
    X = 'NA';
    Z = 'NA';
else
    disp('Phase II to determine optimal basic feasible solution ')
    A_refined = (A_return(1:size(A,1), 1: size(A,2)));
    b_refined = b_return;
    c_refined = c;
    basic_variables = basic_variables(1:size(A,2));
    [A, b,basic_variables,D,flag_new] =
revised_simplex_iteration(A_refined,b_refined,c_refined,basic_variables);
    if flag_new == 0
        X = 'NA';
```

```
        Z = 'NA';

    else
        % Find columns of A that are basic.
        X = zeros(1,size(A,2));
        for i = 1 : size(A,2)
            if basic_variables(i) == 1
                X(i) = b(A(:,i) == 1);
            end
        end
        Z = c_refined*X(:);
        disp('******************************************************** ')
        display(sprintf(' The optimal solution is given by: X : %s \n',num2str(X)))
        display(sprintf(' The optimal objective value is given by: Z : %d \n',Z))
        disp('********************************************************')

    end
end
```

Part – 2

```
%Function coded by Immanuel Manohar (idmanohar@crimnson.ua.edu)

%Performs Revised simplex iterations
function [A_return, b_return, basic_variables,y, flag] = ...
    revised_simplex_iteration(A,b,c,basic_variables)
flag = 1;
b = b(:);
c= c(:)';
basic_variables = basic_variables(:)';
D = (basic_variables == 0);
CD = c(D==1);
CB = c(basic_variables==1);
B = A(:,basic_variables==1);
AD = A(:,D);
y = CB*(B^-1);
RD = CD - y*AD;
a0_bar = B^-1*b;
Iteration = 1;

basic_row = round(zeros(size(basic_variables)));
basic_row(basic_variables ==1) = round(sum(diag(1 : size(B,2))*B)); %stores which row
has the non-zero component in the basis matrix B.

display('Initial Tableau:')
basics = find(basic_variables==1);
non_basics = find(D==1);
Tableau = [basics(:) B^-1  a0_bar(:)]

while (Iteration < 20)

    if sum(RD<0) == 0
        display('All relative cost coeffitients > 0, optimal solution reached')
        A_return = (B^-1)*A;
        b_return = a0_bar;

        return
    else
        [~,ID] = min(RD);
        pivot_column = non_basics(ID(1));
       % AD = A(:,D);
```

```
        aq = A(:,pivot_column);
        aq_bar = (B^-1)*aq; %aq  interms of current basis

        if sum(aq_bar > 0) == 0
            display('The problem is unbounded')
            A_return = A;
            b_return = b;
            flag = 0;
            return
        else
            ratio = a0_bar./aq_bar;
            [~,Id] = min((ratio > 0).*ratio + (ratio <= 0).*Inf);
            pivot_row = Id(1);
            fprintf('Pivot Row: %d; Pivot Column: %d \n',pivot_row,pivot_column)

            column_replace = find(basic_row == pivot_row);


            basic_variables(column_replace) = 0;
            basic_variables(pivot_column) = 1;
            basic_row(column_replace) = 0;
            basic_row(pivot_column) = pivot_row;

            B = A(:,basic_variables==1);

            D = (basic_variables == 0);
            fprintf('%d Tableau:', Iteration)
            AD = A(:,D);
            basics = find(basic_variables==1);
            non_basics = find(D==1);

            a0_bar = B^-1*b;
            Tableau = [basics(:) B^-1  a0_bar(:)]
            Iteration = Iteration +1;
            CD = c(D==1);
            CB = c(basic_variables==1);
            y = CB*(B^-1);
            RD = CD - y*AD;
        end
    end
end
```

The Dual Simplex:

The following function implements the dual simplex method:

```
%function coded by Immanuel Manohar (idmanohar@crimson.ua.edu)
%inspired by codes by Kaushik (A friendly collegue)
%Dual simplex helps avoid phase I of finding an Initial B.F.S.
%solves
%min c'*x
%st
%Ax <= b;
% x>=0;
function [X,Z,flag] = Dual_simplex(fname)
flag = 1;
load(fname)

b = b(:); % converting the b into column matrix b=transpose(b)
[m, n] = size(A); % n -> Number of Primal variables, m -> Number of Dual Variables
```

```
eq = 0;
if length(c) < n % length c less than numver of primal variables.
    c = [c zeros(1,n-length(c))]; % Assign zeros to cost of extra primal variables for
which cost was not specified
end



A=[A eye(m)]; % adding the identity matrix for whole A
ncol = size(A,2); % length of Matrix A
c = [c zeros(1,ncol-length(c))]; % make the length of c equal to ncol of A
Tableau = [A b; c 0];
subs = ncol+1:ncol+m; % this indicates column number which has identity matrix possess
as basic variables

disp(sprintf('\n\n Initial tableau'))
Tableau
A = Tableau;
[bmin, row] = min(b); % to find the min b of RHS in Ax>=b
tbn = 0; % initialize the number of tableau
%Dual-simplex operation% Pivot operation
while ~isempty(bmin) && bmin < 0 && abs(bmin) > eps
    if A(row,1:m+eq+n) >= 0 % checking whether the row particular to negative b is not
positive
     disp(sprintf('\n Empty feasible region \n'))
%     varargout(1)={subs(:)}; % if the row is positive , position of the basic
variables
%     varargout(2)={A}; % final A value
%     varargout(3) = {zeros(n,1)}; % flush all the basic variables to zeros
%     varargout(4) = {0}; % final cost value set as zero
      X = 'NA'
      Z = 'NA'
      flag = 0;

    return
    end
    aa = A(m+1,1:m+n);
    bb = A(row,1:m+n);
    bb1 = bb + (bb == 0).*1;
    [~,col] = max(((aa./bb1).*(bb<0))+(bb>=0).*-10000); %to find the pivot element
    disp(sprintf(' pivot row-> %g pivot column-> %g',row,col))
    subs(row) = col; % the pivot element gets added to basic
    A(row,:)= A(row,:)/A(row,col); % making pivot element equal to 1
    for i = 1:m+eq+1 % Update the tableau
        if i ~= row
        A(i,:)= A(i,:)-A(i,col)*A(row,:); % pivot operation
        end
    end
    tbn = tbn + 1; %update the number of tableau
    disp(sprintf('\n\n Tableau %g',tbn))
    Tableau = A
[bmin, row] = min(A(1:m+eq,m+eq+n+1)); % i.e process of finding pivot column
end
x = zeros(m+n+eq,1); % flushing out the old values stored for the variable x
x(subs) = A(1:m+eq,m+eq+n+1); %% Updating the final basic variable values
x = x(1:n);

z = -A(m+eq+1,n+m+eq+1);
X = x;
Z = z;
```

Transportation Problems:

The following function converts the unbalanced transportation problem to a balanced if necessary and hence solves both the unbalanced and balanced transportation problems.

Part 1

```
function [X,Z] = simplex_transportaion(fname)

load(fname);

if sum(a) < sum(b)
    display('infeasible as demands cannot be met with the sources');
elseif sum(a) == sum(b)
    [X,Z] = simplex_balancedtransportation(name);
elseif sum(a) > sum(b)
    if  (isempty(S) == 1) || (sum(S ~= 0) == 0)
    %create a single dummy demand node
    new_demand = sum(a) - sum(b);
    b = [b new_demand];
    C = [ C  zeros(size(C,1),1)];
    else
    %create a single dummy demand node
    new_demand = sum(a) - sum(b);
    b = [b new_demand];
    mm = min(S);
    C = [ C  ones(size(C,1),1).*mm(1)];
    end
    save(name,'C','a','b','S','Type','name');
    [X,Z] = simplex_balancedtransportation(name);
end
```

Part 2

This following function solves the Balanced transportation problem created in the previous function block:

```
%funciton coded by Immanuel Manohar (idmanohar@crimson.ua.edu)

function [X,Z] = simplex_balancedtransportation(fname)

load(fname)
X = zeros(length(a), length(b));

eps = 10^-5;
%% Implementing northwestcorner rule to get Initial BFS.
col = 1;
row = 1;
a_temp = a;
b_temp = b;
while ((col <= length(b)) && (row <= length(a)))
    X(row,col) = min(a_temp(row),b_temp(col));
    rem = a_temp(row)-b_temp(col);
    if rem > 0
        col = col +1;
        a_temp(row) = rem;
    elseif rem < 0
```

```
        row = row +1;
        b_temp(col) = -rem;
    else
        if ((col <= length(b)) && (row <= length(a)))
        row = row+1;
        X(row,col) = eps;
        col = col+1;
        end
    end
end
display('Initial BFS obtained using North West Corner Rule:')
X = X(1:length(a), 1: length(b))

%%
%  a
%  b

%computing cost:
Iteration = 0;
while (1)
        Iteration = Iteration +1;
         size(C)
         size(X)
    Cost_1 = C.*(X > 0);

    u = ones(size(b))*NaN;
    v = ones(size(a))*NaN;
    v(end) = 0;

    idx = find(X>0);
    [I,J] = ind2sub(size(C),idx);
%     o = 0;
%     while (o < 3)
        for i = length(I) : -1 : 1

            if isnan(u(J(i)))
                u(J(i)) = -v(I(i))+C(I(i),J(i));
            elseif isnan(v(I(i)))
                v(I(i)) = -u(J(i))+C(I(i),J(i));
            end
        end
%         o = o+1;
%     end

     u
     v
    display(sprintf('The %d tableau: ',Iteration))
    Tableau = [Cost_1 v'; u 0]

    Relative_cost =    C - (diag(v)*ones(size(C)) + ones(size(C))*diag(u))
    R = Relative_cost;


        if sum(sum(R<0))  == 0
            display( ' Optimal Solution Reached')
            Z = sum(sum(Cost_1.*X));
            return
        else
            idx = find( R == min(min((R))));
            [i,j] = ind2sub(size(C),idx(1));
            theta = min([X(i,:),X(:,j)']);
            [x_new] = cycle(X,i,j,X>0,R);
```

```
            X = x_new;
        end
end
```

Auxiliary:

The following function helps in updating the basic feasible solution with the introduction of a new xij:

```
%Editted by: Immanuel Manohar (idmanohar@crimson.ua.edu)
%function inspired by my collegue Kaushik's algorithm

function [y]=cycle(x,row,col,b,Rd)
% [y,bout]=cycle(x,row,col)
% x: current solution (a m*n matrix)
% b: entering basic variables (m*n)
% row,col: index for element entering basis
% y: solution after cycle of change (m*n)

    y=x;
    [m,n]=size(x);
    loop=[row col]; % describes the cycle of change
    disp('Element entering the basic')
    fprintf('R(%d,%d)= %d\n\n',loop(1,1),loop(1,2),Rd(loop(1,1),loop(1,2)))
    x(row,col)=Inf; % do not include (row,col) in the search
    b(row,col)=Inf;
    rowsearch=1; % start searching in the same row
    while (loop(1,1)~=row | loop(1,2)~=col | length(loop)==2),
        if rowsearch, % search in row
        j=1;
        while rowsearch
        if (b(loop(1,1),j)~=0) & (j~=loop(1,2)) % element is part of basic solution
and not in the same column
        loop=[loop(1,1) j ;loop]; % add indices of found element to loop
        rowsearch=0; % start searching in columns
        elseif j==n, % no interesting element in this row
        b(loop(1,1),loop(1,2))=0;
        loop=loop(2:length(loop),:); % backtrack
        rowsearch=0;
        else
        j=j+1; % update the column
        end
        end
        else % column search to find the elements which get affected by addition of
new basic element
        i=1;
        while ~rowsearch
        if (b(i,loop(1,2))~=0) & (i~=loop(1,1)) % if the element is part of basic
solution and not in the same row
        loop=[i loop(1,2) ; loop]; % add the found element to loop to estimate theta
        rowsearch=1; % start searching in rows
        elseif i==m
        b(loop(1,1),loop(1,2))=0;
        loop=loop(2:length(loop),:); % backward
        rowsearch=1;
        else
        i=i+1; % update the row
        end
        end
        end
```

```
    end
disp('Step-3')
% compute maximal loop shipment
l=length(loop);
theta=Inf;
minindex=Inf;
for i=2:2:l
if x(loop(i,1),loop(i,2))<theta,
theta=x(loop(i,1),loop(i,2)); % calculating the minimum theta value
minindex=i;
end;
end
% compute new transport matrix
y(row,col)=theta;
disp('Update Tableu')
for i=2:l-1
y(loop(i,1),loop(i,2))=y(loop(i,1),loop(i,2))+(-1)^(i-1)*theta;
end
disp('x(updated) =')
disp(y)
```

Test Results:

Example 1:

```
fname =

example_1.mat

Number of optimization variables: 3
Number of basic variables: 2
Phase I to determine initial basic feasible solution
The initial simplex tableau is given by:

Tableau =

     2     1     2     1     0     4
     3     3     1     0     1     3
     0     0     0     1     1     0

The 1st tableau after transforming the last row is:

Tableau =

     2     1     2     1     0     4
     3     3     1     0     1     3
    -5    -4    -3     0     0    -7

Pivot Row: 1; Pivot Column: 1
The 2 tableau is:


Tableau =

    1.0000    0.5000    1.0000    0.5000         0    2.0000
         0    1.5000   -2.0000   -1.5000    1.0000   -3.0000
         0   -1.5000    2.0000    2.5000         0    3.0000

Pivot Row: 2; Pivot Column: 2
The 3 tableau is:


Tableau =

    1.0000         0    1.6667    1.0000   -0.3333    3.0000
         0    1.0000   -1.3333   -1.0000    0.6667   -2.0000
         0         0         0    1.0000    1.0000         0
```

```
All relative cost coeffitients > 0, optimal solution reached
Phase II to determine optimal basic feasible solution
The initial simplex tableau is given by:

Tableau =

    1.0000         0    1.6667    3.0000
         0    1.0000   -1.3333   -2.0000
    4.0000    1.0000    1.0000         0

The 1st tableau after transforming the last row is:

Tableau =

    1.0000         0    1.6667    3.0000
         0    1.0000   -1.3333   -2.0000
         0         0   -4.3333  -10.0000

Pivot Row: 1; Pivot Column: 3
The 2 tableau is:


Tableau =

    0.6000         0    1.0000    1.8000
    0.8000    1.0000         0    0.4000
    2.6000         0         0   -2.2000

All relative cost coeffitients > 0, optimal solution reached
*********************************************************
 The optimal solution is given by: X : 0         0.4         1.8

 The optimal objective value is given by: Z : 2.200000e+00

*********************************************************
-------------------------
The optimal basic feasible solution(s) is:

X =

         0    0.4000    1.8000
```

```
-------------------------
The optimal cost is Z : 2.200000e+00

Z =

    2.2000

The dual is:

D =

NA

---------(Results are also stored in the file name: example_1.mat)---------------->>
```

Example 2:

```
fname =

Example_2.mat

Number of optimization variables: 3
Number of basic variables: 2
Phase I to determine initial basic feasible solution
Initial Tableau:

Tableau =

     4     1     0     4
     5     0     1     3

Pivot Row: 1; Pivot Column: 1
1 Tableau:
Tableau =

    1.0000    0.5000         0    2.0000
    5.0000   -1.5000    1.0000   -3.0000

Pivot Row: 2; Pivot Column: 2
2 Tableau:
Tableau =

    1.0000    1.0000   -0.3333    3.0000
    2.0000   -1.0000    0.6667   -2.0000

All relative cost coeffitients > 0, optimal solution reached
Phase II to determine optimal basic feasible solution
Initial Tableau:

Tableau =

    1.0000    1.0000    0.0000    3.0000
    2.0000    0.0000    1.0000   -2.0000

Pivot Row: 1; Pivot Column: 3
1 Tableau:
Tableau =

    2.0000    0.8000    1.0000    0.4000
    3.0000    0.6000    0.0000    1.8000
```

```
All relative cost coeffitients > 0, optimal solution reached
*********************************************************
 The optimal solution is given by: X : 0         0.4         1.8

 The optimal objective value is given by: Z : 2.200000e+00

*********************************************************
-------------------------
The optimal basic feasible solution(s) is:

X =

         0    0.4000    1.8000

-------------------------
The optimal cost is Z : 2.200000e+00

Z =

    2.2000

The dual is:

D =

    1.4000    1.0000

---------(Results are also stored in the file name: Example_2.mat)---------------->>
```

Example 3:

```
fname =

Example_4.mat


 Initial tableau

Tableau =

    -1    -2    -3     1     0    -5
    -2    -2    -1     0     1    -6
     3     4     5     0     0     0

 pivot row-> 2 pivot column-> 1


 Tableau 1

Tableau =

         0   -1.0000   -2.5000    1.0000   -0.5000   -2.0000
    1.0000    1.0000    0.5000         0   -0.5000    3.0000
         0    1.0000    3.5000         0    1.5000   -9.0000

 pivot row-> 1 pivot column-> 2


 Tableau 2

Tableau =

         0    1.0000    2.5000   -1.0000    0.5000    2.0000
    1.0000         0   -2.0000    1.0000   -1.0000    1.0000
         0         0    1.0000    1.0000    1.0000  -11.0000

--------------------------
The optimal basic feasible solution(s) is:

X =

     1
     2
--------------------------
The optimal basic feasible solution(s) is:

X =

     1
     2
     0

--------------------------
The optimal cost is Z : 11

Z =

    11

The dual is:

D =

NA

---------(Results are also stored in the file name: Example_4.mat)----------------->>
```

Example 4:

```
fname =

Example_3.mat

Initial BFS obtained using North West Corner Rule:

X =

    10    20     0     0     0
     0    30    20    30     0
     0     0     0    10     0
     0     0     0    40    20

The 1 tableau:

Tableau =

     3     4     0     0     0     3
     0     2     4     5     0     1
     0     0     0     3     0    -1
     0     0     0     4     2     0
     0     1     3     4     2     0


Relative_cost =

     0     0     0     1     4
     1     0     0     0     2
     3     2     0     0     1
     3     2    -1     0     0

Element entering the basic
R(4,3)= -1

Step-3
Update Tableu
x(updated) =
    10    20     0     0     0
     0    30     0    50     0
     0     0     0    10     0
     0     0    20    20    20
```

```
The 2 tableau:

Tableau =

     3     4     0     0     0     3
     0     2     0     5     0     1
     0     0     0     3     0    -1
     0     0     2     4     2     0
     0     1     2     4     2     0


Relative_cost =

     0     0     1     1     4
     1     0     1     0     2
     3     2     1     0     1
     3     2     0     0     0

 Optimal Solution Reached
-------------------------
The optimal basic feasible solution(s) is:

X =

    10    20     0     0     0
     0    30     0    50     0
     0     0     0    10     0
     0     0    20    20    20

-------------------------
The optimal cost is Z : 610

Z =

   610

---------(Results are also stored in the file name: Example_3.mat)----------------->>
```

Example 5

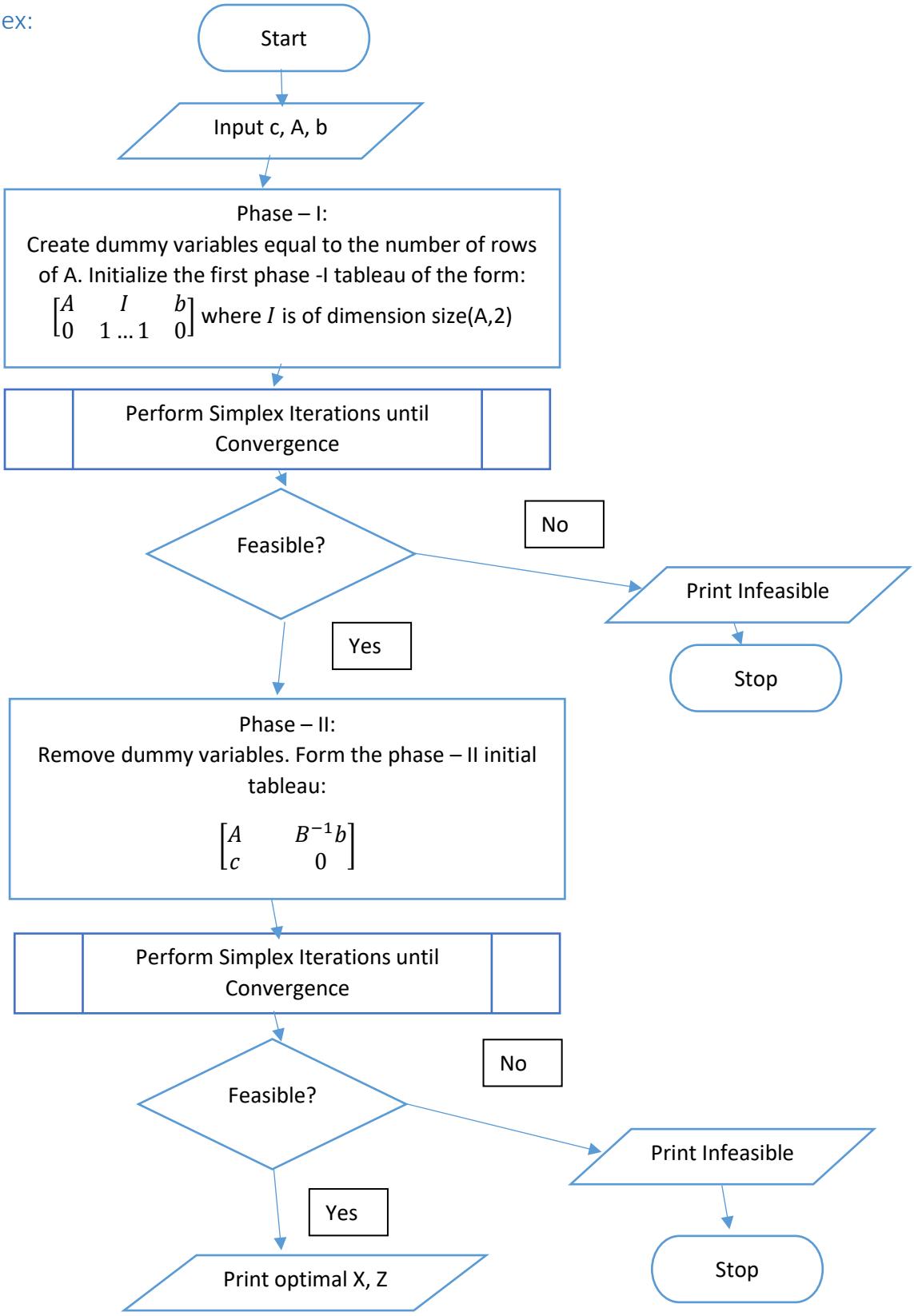
```
You are trying to solve a transportation problem
Enter the input parametrs
-------------------------
Enter the cost matrix,c: [4 6 8 9;2 4 5 5;2 2 3 2;3 2 4 2]
Enter the source quantities: [30 80 10 60]
Enter the destination quantities: [10 50 20 80]
Enter the storage cost (if any): [0 0 0 0]
Input parameters are entered
----------------------------
solution to be found using Array format of simplex based on northwest corner rule
The cost c is given by:

C =

     4     6     8     9
     2     4     5     5
     2     2     3     2
     3     2     4     2

The source quantities is given by:

a =

    30    80    10    60

The Destination quantities is given by:

b =

    10    50    20    80

The Storage cost is given by:

S =

     0     0     0     0

confirm above and store values? (1 = yes, 0 = no)1
enter file name to store as:Example_5
Initial BFS obtained using North West Corner Rule:
```

```
X =

    10    20     0     0     0
     0    30    20    30     0
     0     0     0    10     0
     0     0     0    40    20

The 1 tableau:

Tableau =

     4     6     0     0     0     5
     0     4     5     5     0     3
     0     0     0     2     0     0
     0     0     0     2     0     0
    -1     1     2     2     0     0


Relative_cost =

     0     0     1     2    -5
     0     0     0     0    -3
     3     1     1     0     0
     4     1     2     0     0

Element entering the basic
R(1,5)= -5

Step-3
Update Tableu
x(updated) =
    10     0     0     0    20
     0    50    20    10     0
     0     0     0    10     0
     0     0     0    60     0
```

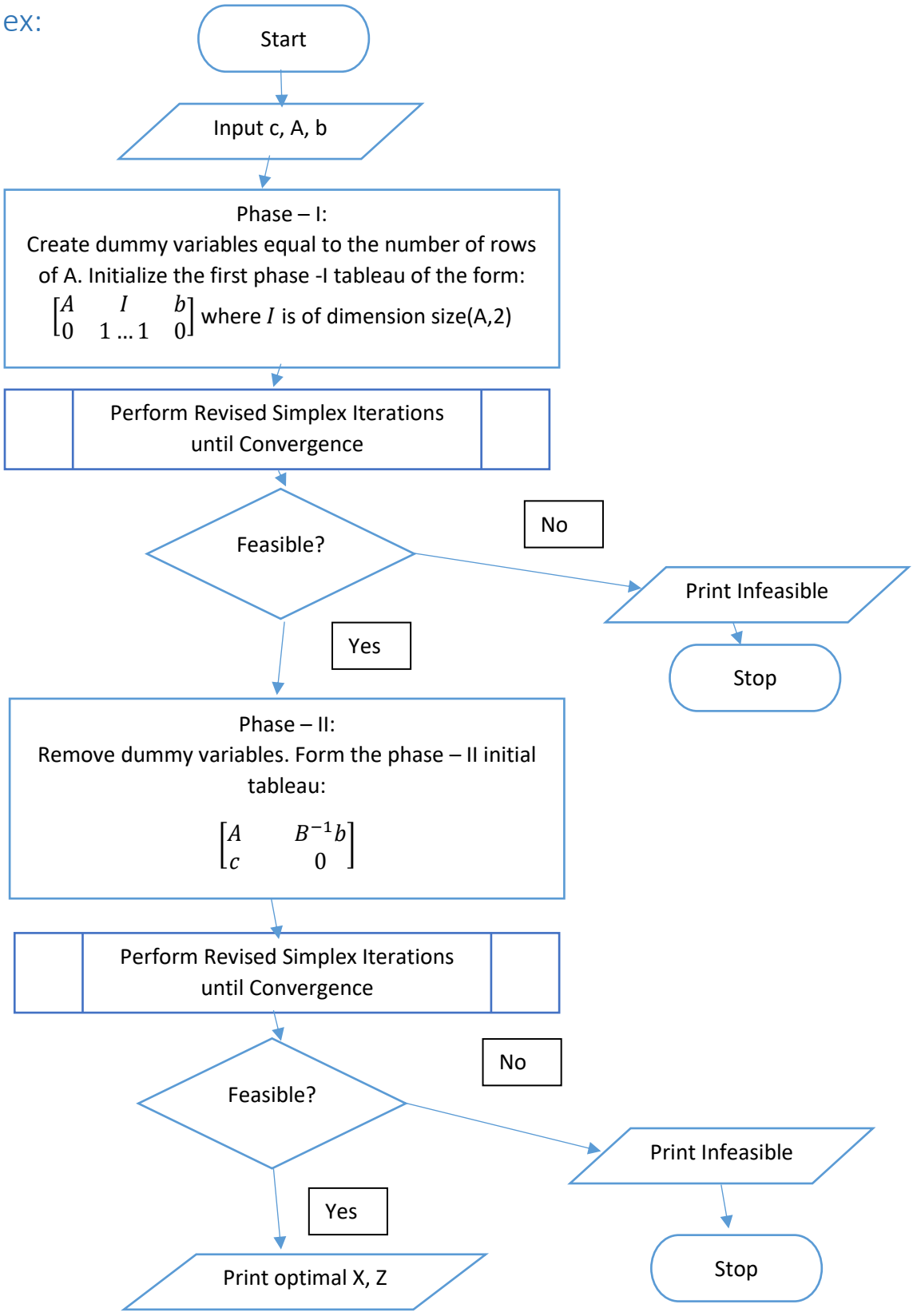
```
The 2 tableau:

Tableau =

     4     0     0     0     0   NaN
     0     4     5     5     0     3
     0     0     0     2     0     0
     0     0     0     2     0     0
   NaN     1     2     2   NaN     0


Relative_cost =

   NaN   NaN   NaN   NaN   NaN
   NaN     0     0     0   NaN
   NaN     1     1     0   NaN
   NaN     1     2     0   NaN

 Optimal Solution Reached
-------------------------
The optimal basic feasible solution(s) is:

X =

    10     0     0     0    20
     0    50    20    10     0
     0     0     0    10     0
     0     0     0    60     0

-------------------------
The optimal cost is Z : 530

Z =

   530

---------(Results are also stored in the file name: Example_5.mat)----------------->>
```

Flowcharts for the different function blocks:

Two Phase Simplex:



Revised Simplex:



Dual Simplex: